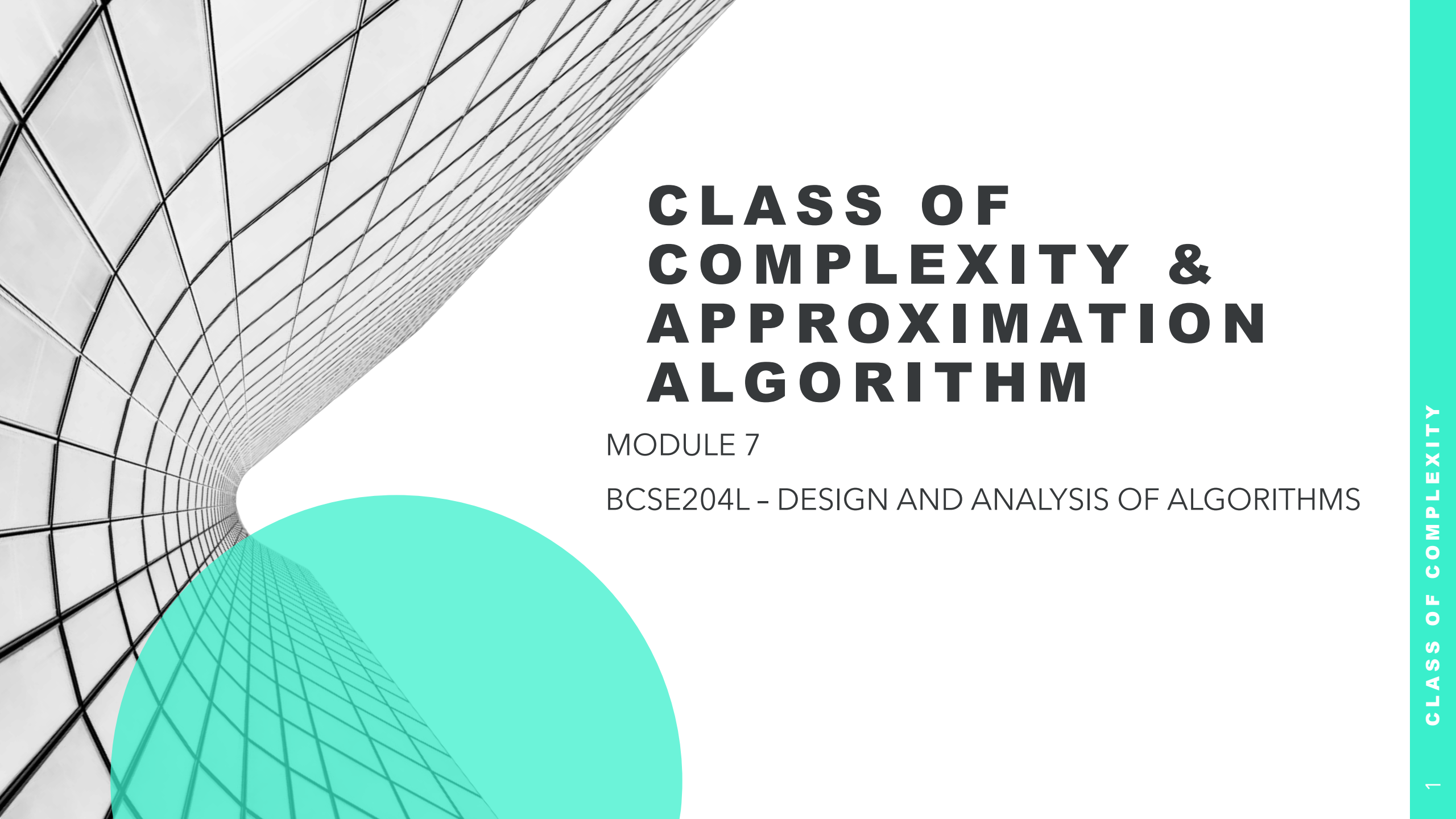




CLASS OF COMPLEXITY & APPROXIMATION ALGORITHM

MODULE 7

BCSE204L - DESIGN AND ANALYSIS OF ALGORITHMS



APPROXIMATION ALGORITHMS

INTRODUCTION

Approximation algorithms are used to get a solution close to the (optimal) solution of an optimization problem in polynomial time

For example, Traveling Salesman Problem (TSP) is an optimization problem (the route taken has to have minimum cost).

Since TSP is NP-Complete, approximation algorithms allow for getting a solution close to the solution of an NP problem in polynomial time.

DEFINITION

An algorithm is an *α -approximation algorithm* for an optimization problem Π if

1. The algorithm runs in polynomial time
2. The algorithm always produces a solution that is within a factor of α of the optimal solution

Minimization - $\alpha < 1$

Maximization - $\alpha > 1$

WHY APPROXIMATION ALGORITHMS

Problems that we cannot find an optimal solution in a polynomial time

Eg: Set Cover, Bin Packing

Need to find a near-optimal solution:

Heuristic

Approximation algorithms:

This gives us a guarantee approximation ratio

APPROXIMATION ALGORITHMS

An algorithm that returns **near-optimal** solutions in polynomial time

Approximation Ratio $\rho(n)$:

Define: C^* as a optimal solution and C is the solution produced by the approximation algorithm

$$\max (C/C^*, C^*/C) \leq \rho(n)$$

Maximization problem: $0 < C \leq C^*$, thus C^*/C shows that C^* is larger than C by $\rho(n)$

Minimization problem: $0 < C^* \leq C$, thus C/C^* shows that C is larger than C^* by $\rho(n)$

APPROXIMATION ALGORITHMS (CONT)

PTAS (Polynomial Time Approximation Scheme): A $(1 + \epsilon)$ -approximation algorithm for a NP-hard optimization Π where its running time is bounded by a polynomial in the size of instance I

FPTAS (Fully PTAS): The same as above + time is bounded by a polynomial in both the size of instance I and $1/\epsilon$

A DILEMMA!

We cannot find C^* , how can we compare C to C^* ?

How can we design an algorithm so that we can compare C to C^*

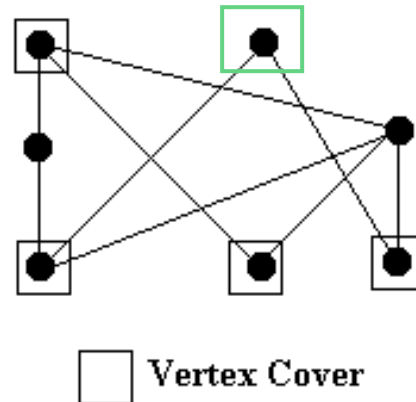
It is the objective of this course!!!

VERTEX COVER

Definition:

Given an undirected graph $G = (V, E)$, a subset $V' \subseteq V$ is called a vertex cover of G iff for every edge $e \in E$, e has at least one endpoint incident at V'

An Example



VERTEX COVER PROBLEM

Definition:

Given an undirected graph $G=(V,E)$, find a vertex cover with minimum size (has the least number of vertices)

This is sometimes called cardinality vertex cover

More generalization:

Given an undirected graph $G=(V,E)$, and a cost function on vertices $c: V \rightarrow \mathbb{Q}^+$

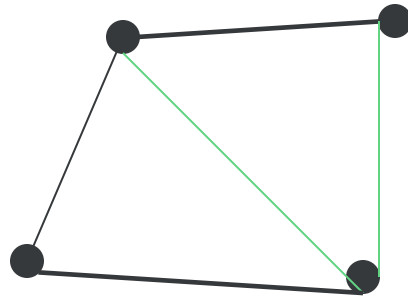
Find a minimum cost vertex cover

HOW TO SOLVE IT

Matching:

A set M of edges in a graph G is called a **matching** of G if no two edges in set M have an endpoint in common

Example:



HOW TO SOLVE IT (CONT)

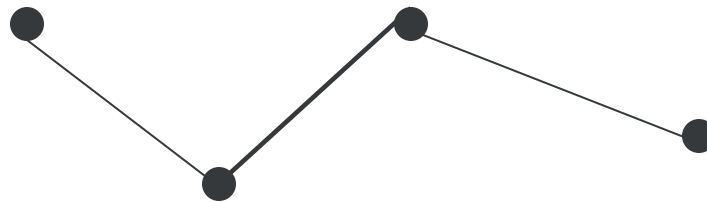
Maximum Matching:

A matching of G with the greatest number of edges

Maximal Matching:

A matching which is not contained in any larger matching

Note: Any maximum matching is certainly maximal, but not the reverse

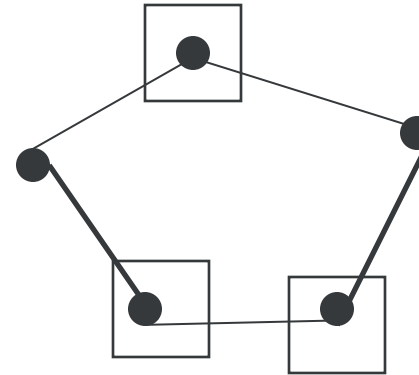
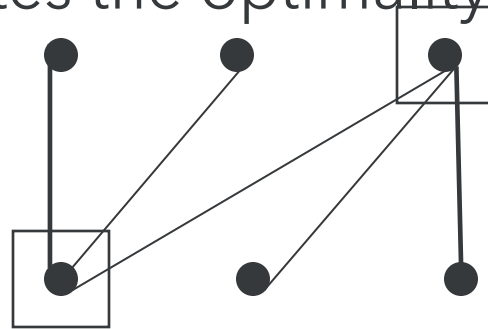


MAIN OBSERVATION

No vertex can cover two edges of a matching

The size of every vertex cover is at least the size of every matching: $|M| \leq |C|$

$|M| = |C|$ indicates the optimality



Possible Solution: Using Maximal matching to find Minimum vertex cover

AN ALGORITHM

Alg 1: Find a maximal matching in G and output the set of matched vertices

Theorem: The algorithm 1 is a factor 2-approximation algorithm.

Let opt be a size of an optimal solution. Let C be a set of

Proof: vertex cover obtained from algorithm 1. We have:

$$|M| \leq opt \text{ and } |C| = 2|M|$$

Therefore, $|C| \leq 2opt$

CAN ALG1 BE IMPROVED?

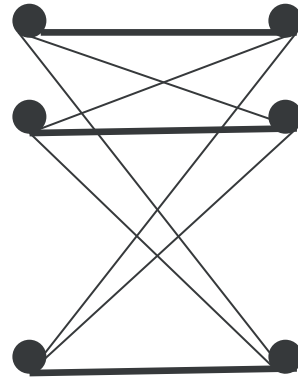
Q1: Can the approximation ratio of Alg 1 be improved by a better analysis?

Q2: Can we design a better approximation algorithm using the similar technique (maximal matching)?

Q3: Is there any other lower bounding method that can lead to a better approximation algorithm?

ANSWERS

A1: No by considering the complete bipartite graphs $K_{n,n}$



A2: No by considering the complete graph K_n where n is odd.

$$|M| = (n-1)/2 \text{ whereas } \text{opt} = n - 1$$

ANSWERS (CONT)

A3:

Currently a central open problem

Yes, we can obtain a better one by using the semidefinite programming

Generalization vertex Cover

Can we still able to design a 2-approximation algorithm?

Homework assignment!

SET COVER

Definition: Given a universe U of n elements, a collection of subsets of U , $\mathcal{S} = \{S_1, \dots, S_m\}$, and a cost function $c: \mathcal{S} \rightarrow \mathbb{Q}^+$, find a minimum cost subcollection C of $\mathcal{S}$ that covers all elements of U .

Example:

$$U = \{1, 2, 3, 4, 5\}$$

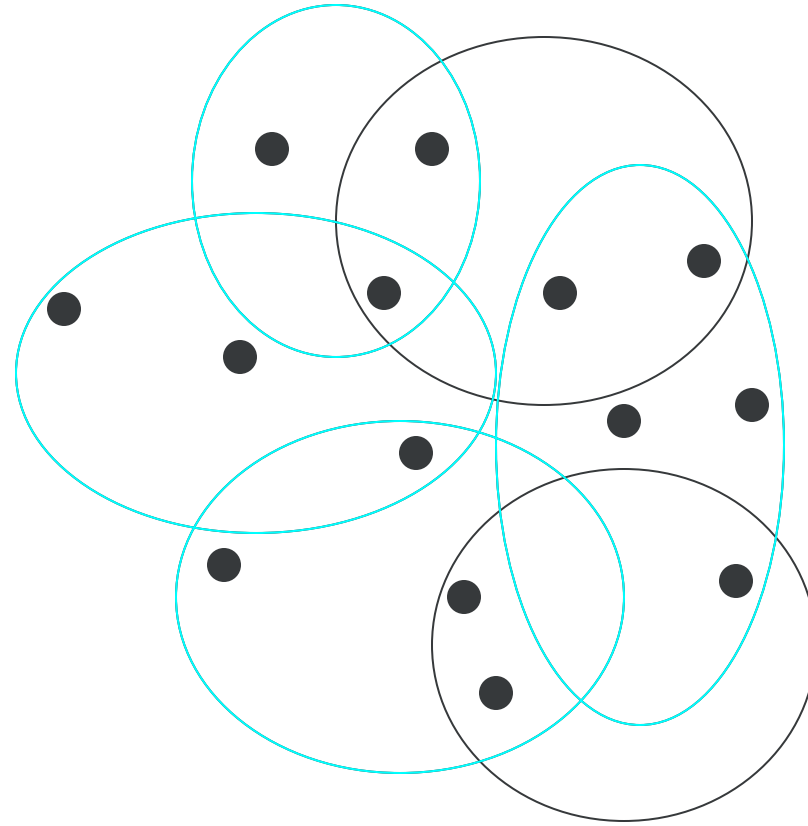
$$S_1 = \{1, 2, 3\}, S_2 = \{2, 3\}, S_3 = \{4, 5\}, S_4 = \{1, 2, 4\}$$

$$c_1 = c_2 = c_3 = c_4 = 1$$

$$\text{Solution } C = \{S_1, S_3\}$$

If the cost is uniform, then the set cover problem asks us to find a subcollection covering all elements of U with the minimum size.

AN EXAMPLE



NP-COMPLETENESS

Theorem: Set Cover (SC) is NP-complete

Proof:

INSTANCE: Given a universe U of n elements, a collection of subsets of U , $\mathcal{S} = \{S_1, \dots, S_m\}$, and a positive integer b

QUESTION: Is there a set $C \subseteq \{1, 2, \dots, m\}$, $|C| \leq b$, such that

$$\bigcup_{i \in C} S_i = U.$$

(Note: The subcollection $\{S_i \mid i \in C\}$ satisfying the above condition is called a set cover of U)

PROOF (CONT)

First we need to show that SC is in NP. Given a collection of sets C , it is easy to verify that if $|C| \leq b$ and the union of all sets listed in C does include all elements in U .

To complete the proof, we need to show that Vertex Cover (VC) $\leq_p$ Set Cover (SC)

Given an instance C of VC (an undirected graph $G=(V,E)$ and a positive integer j), we need to construct an instance C' of SC in polynomial time such that C is satisfiable iff C' is satisfiable.

PROOF (CONT)

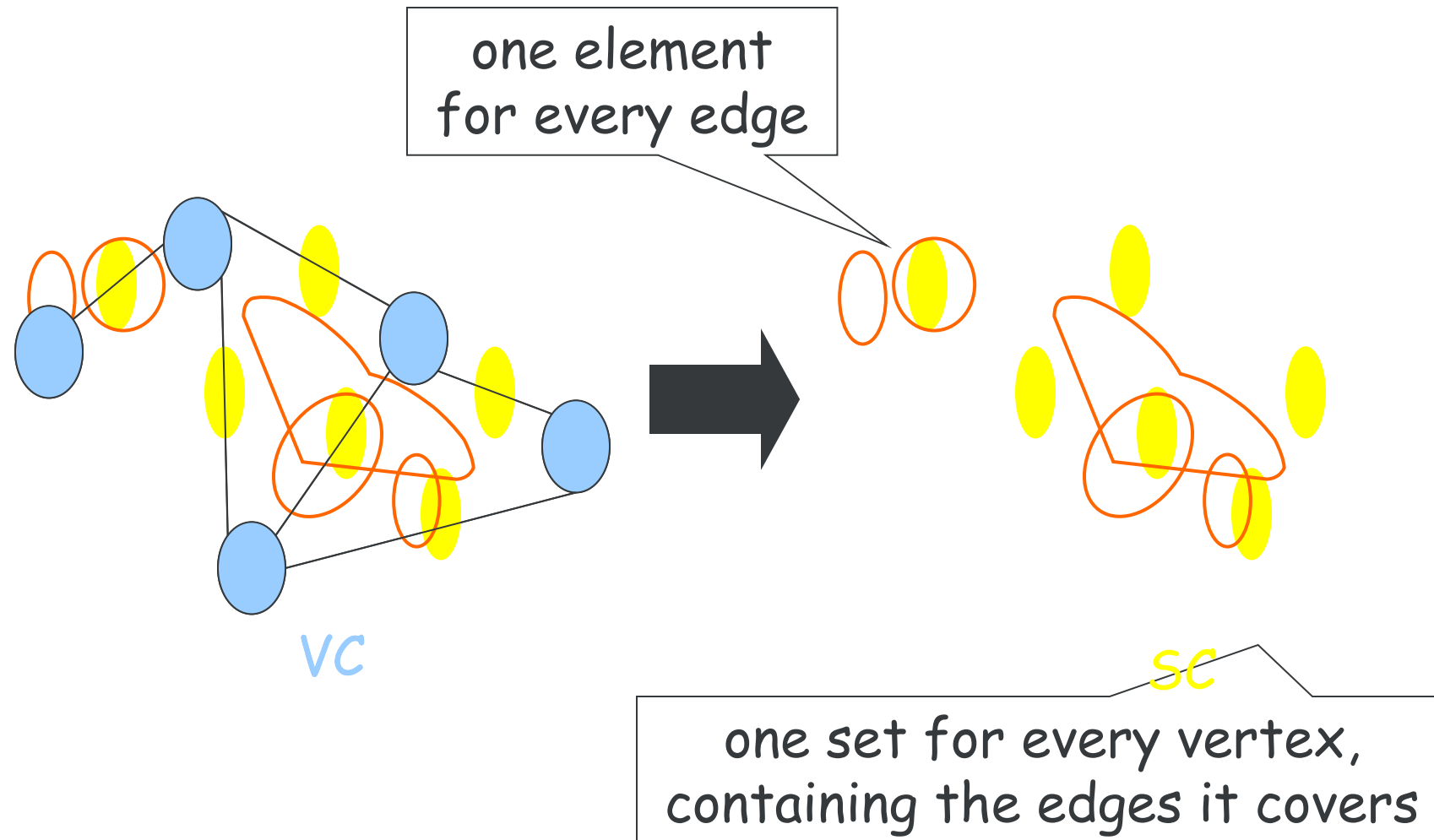
Construction: Let $U = E$. We will define n elements of U and a collection $\mathcal{S}$ as

follows:

Label all vertices in V from 1 to n . Let S_i be the set of all edges that incident to vertex i . Finally, let $b = j$. This construction is in poly-time with respect to the size of VC instance.

Note: Each edge corresponds to each element in U and each vertex corresponds to each set in $\mathcal{S}$.

VERTEX-COVER $\leq_p$ SET-COVER



PROOF (CONT)

Now, we need to prove that C is satisfiable iff C' is. That is, we need to show that if the original instance of VC is a yes instance iff the constructed instance of SC is a yes instance.

($\rightarrow$) Suppose G has a vertex cover of size at most j , called C . By our construction, C corresponds to a collection C' of subsets of U . Since $b = j$, $|C'| \leq b$. Plus, C' covers all elements in U since C "covers" all edges in G . To see this, consider any element of U . Such an element is an edge in G . Since C is a set cover, at least endpoint of this edge is in C .

($\leftarrow$) Suppose there is a set cover C' of size at most b in our constructed instance. Since each set in C' is associated with a vertex in G , let C be the set of these vertices. Then $|C| = |C'| \leq b = j$. Plus, C is a vertex cover of G since C' is a set cover. To see this, consider any edge e . Since e is in U , C' must contain at least one set that includes e . By our construction, the only set that include e correspond to nodes that are endpoints of e . Thus C must contain at least one endpoints of e .

SOLUTIONS

Algorithm 1: (in the case of uniform cost)

1: $C = \text{empty}$

2: **while** U is not empty

3: pick a set S_i such that S_i covers **the most elements** in U

4: remove the new covered elements from U

5: $C = C \text{ union } S_i$

6: **return** C

SOLUTIONS (CONT)

In the case of non-uniform cost

Similar method. At each iteration, instead of picking a set S_i such that S_i covers the most uncovered elements, we will pick a set S_i whose cost-effectiveness α is smallest, where α is defined as:

$$\alpha = c(S_i) / |S_i \cap U|$$

Questions: Why choose smallest α ? Why define α as above

SOLUTIONS (CONT)

Algorithm 2: (in the case of non-uniform cost)

- 1: $C = \text{empty}$
- 2: **while** U is not empty
- 3: pick a set S_i such that S_i has the smallest α
- 4: for each new covered elements e in U
- 5: set **price**(e) = α
- 6: remove the new covered elements from U
- 7: $C = C \text{ union } S_i$
- 8: **return** C

APPROXIMATION RATIO ANALYSIS

Let $e_k, k = 1 \dots n$, be the elements of U in the order in which they were covered by Alg 2. We have:

Lemma 1: For each $k \in \{1, \dots, n\}$, $\text{price}(e_k) \leq \text{opt} / (n - k + 1)$
where opt is the cost of the optimal solution

Proof: Let U_j be the set the remaining set U at iteration j . That is, U_j contains all the uncovered elements at iteration j . At any iteration j , the leftover sets of the optimal solution can cover the remaining elements at a cost of at most opt . (Why?)

PROOF OF LEMMA 1 (CONT)

Thus, among these leftover sets, there must exist one set where its α value is at most $opt/|U_j|$. Let e_k be the element covered at this iteration, $|U_j| \geq n - k + 1$. Since e_k was covered by the most cost-effective set in this iteration, we have:

$$\text{price}(e_k) \leq opt/|U_j| \leq opt/(n-k+1)$$

APPROXIMATION RATIO

Theorem 1: The set cover obtained from Alg 2 (also Alg 1) has a factor of H_n where H_n is a harmonic series $H_n = 1 + 1/2 + \dots + 1/n$

Proof: It follows directly from Lemma 1

SET COVER (SC)

A set cover of a set T is any collection of subsets of T whose union is T .

The set cover problem: given a weight for each subset, find the set cover which minimizes the total weight

EXAMPLE OF SET COVER

Set $T = \{ 1, 2, 3, 4, 5, 6, 7, 8 \}$

Available subsets:

1. $S1 = \{ 1, 2, 3 \}$ $w1 = 1$
2. $S2 = \{ 2, 7, 8 \}$ $w2 = 2$
3. $S3 = \{ 4, 5, 6, 7 \}$ $w3 = 3$
4. $S4 = \{ 4, 5, 6, 8 \}$ $w4 = 4$

Solutions: $C = \{ S1, S2, S3 \}$, $C = \{ S1, S2, S4 \}$

Optimal Solution is $C = \{ S1, S2, S3 \}$

WEIGHTED VERTEX COVER (WVC)

Here, we try to find a collection of vertices of a graph such that each edge of the graph contains at least one vertex from the collection

Each vertex has a weight and we try to find the collection that minimizes the total weight

EXAMPLE OF VERTEX COVER

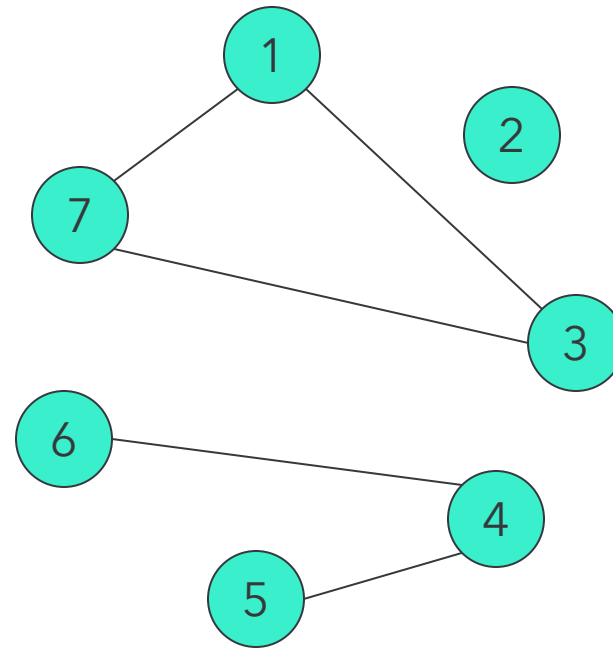
Possible Solutions are:

1. $\{1, 3, 4\}$

2. $\{1, 7, 5\}$

If the weights are the vertex numbers, then the optimal solution is: $\{1, 3, 4\}$

The Graph



VERTEX COVER AS SET COVER

Vertex Cover is a special case of Set Cover

To convert a problem from vertex cover to set cover:

1. Make T = the set of edges E
2. Make the subsets correspond to each vertex, with each subset containing the set of edges that touch the corresponding vertex

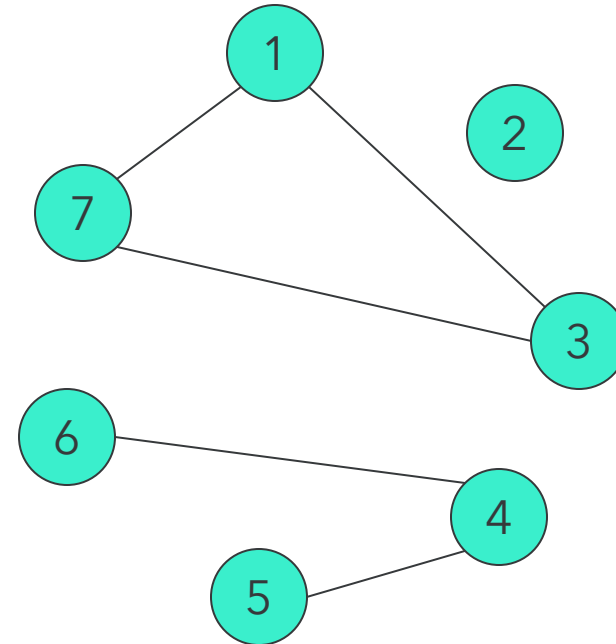
EXAMPLE OF VC AS SC

$T = \{ (1, 3), (3, 7), (1, 7), (4, 5), (4, 6) \}$

Subsets:

1. $S1 = \{ (1, 3), (1, 7) \} w = 1$
2. $S3 = \{ (1, 3), (3, 7) \} w = 3$
3. $S7 = \{ (1, 7), (3, 7) \} w = 7$
4. $S4 = \{ (4, 5), (4, 6) \} w = 4$
5. $S6 = \{ (4, 6) \} w = 6$
6. $S5 = \{ (4, 5) \} w = 5$

The Graph



ALGORITHM FOR UNWEIGHTED SET COVER

$I \leftarrow \emptyset$

while ($T \neq \emptyset$)

 pick $t_i \in T$

$I \leftarrow I \cup \{j : t_i \in S_j\}$

$T \leftarrow T - \bigcup_{j \in I} S_j$

PROOF OF ALGORITHM

Elements are deleted from T only when they are covered, and we delete at least one each time in the loop, at the end of which, no elements are left in T .

Therefore, the algorithm returns a set cover.

DEFINITION OF f

Define $f = \max_{t_i \in T} \left| \{j : t_i \in S_j\} \right|$

In other words, f is the number of subsets in the set cover that cover that element that is in the most subsets in the set cover

For VC problem, $f = 2$ because each element in the set is covered always by 2 subsets (representing the two endpoints of the edge).

ALGORITHM IS F - APPROXIMATION (1)

The Algorithm runs in polynomial time

During each iteration of the loop, a t_j chosen is covered by at least one distinct subset that is in the optimal solution.

This can be shown by contradiction: If there was a t_k chosen in a later iteration that is also in that same S_j then it would have to remain in T after we chose t_j . But when t_j was chosen all sets that contain it were chosen too and all their elements were removed from T including t_k .

ALGORITHM IS f - APPROXIMATION (2)

Therefore, if there are n iterations through the loop, n is less or equal to the number of sets in the optimal solution (OPT).

Each iteration in the loop we pick at most f sets to add to I .

Therefore, $|I| \leq f * n \leq f * \text{OPT}$.

Therefore, the algorithm is an f -approximation.

However, this algorithm is not designed to solve the weighted problem, which is what we wish to solve.

LINEAR PROGRAMMING (LP)

Linear programming is the problem of optimizing a linear function subject to linear inequality constraints.

The function being optimized is called the *objective function*.

The function with the constraints is called the Linear Program.

Any assignment of variables that satisfies the constraints is called a *feasible solution*.

AN EXAMPLE OF LP

Minimize $7x_1 + x_2 + 5x_3$

Constraints:

$$x_1 - x_2 + 3x_3 \geq 10$$

$$5x_1 + 2x_2 - x_3 \geq 6$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$

$$x_3 \geq 0$$

STANDARD FORM OF LP

All constraints are of type greater or equal in minimization LP, and less or equal in maximization LP.

All variables are constrained to be non negative.

By a simple transformation any linear program can be written as a standard minimization or maximization LP.

UPPER BOUND FOR MINIMIZATION

Any upper bound (for the minimization) can be checked by simply finding a solution that satisfies the equation in less than the upper bound.

$(2, 1, 3)$ is such an example for upper bound 30.

LOWER BOUND FOR MINIMIZATION

To find a lower bound, we can note that $7x_1 + x_2 + 5x_3 \geq x_1 - x_2 + 3x_3 \geq 10$, because all coefficients are greater than the corresponding coefficients and all variables are non-negative.

An even better lower bound can be obtained by doing: $7x_1 + x_2 + 5x_3 \geq (x_1 - x_2 + 3x_3) + (5x_1 + 2x_2 - x_3) \geq 16$

MAXIMIZING LOWER BOUND

The minimum solution of the objective function will be obtained when we find the maximum lower bound for the function.

Particularly, it can be formulated as a linear program:

MAXIMIZATION LP OF LOWER BOUND

Maximize $10y_1 + 6y_2$

Constraints:

$$y_1 + y_2 \leq 7$$

$$-y_1 + 2y_2 \leq 1$$

$$3y_1 - y_2 \leq 5$$

$$y_1 \geq 0$$

$$y_2 \geq 0$$

MIN-MAX (PRIMAL-DUAL) EQUIVALENCY

Primal Problem

$$\begin{aligned} &\text{minimize} && \sum_{j=1}^n c_j x_j \\ &\text{subject to} && \forall_{i \in (1, \dots, m)} \sum_{j=1}^n a_{ij} x_j \geq b_i \\ &&& \forall_{j \in (1, \dots, n)} x_j \geq 0 \end{aligned}$$

Dual Problem

$$\begin{aligned} &\text{maximize} && \sum_{i=1}^m b_i y_i \\ &\text{subject to} && \forall_{j \in (1, \dots, n)} \sum_{i=1}^m a_{ij} y_i \leq c_j \\ &&& \forall_{i \in (1, \dots, m)} y_i \leq 0 \end{aligned}$$

RELATION BETWEEN THE MIN/MAX LPS

The first LP will be called the Primal Program and the second LP will be called the Dual Program.

There is a systematic way of finding the Dual of any Primal. Furthermore, the Dual of the Dual of X is X itself.

By construction, every feasible solution to the dual gives a lower bound on the optimum of the primal.

Also, every feasible solution to the primal gives an upper bound on the optimum of the dual.

LP-DUALITY (1)

Furthermore, if the primal solution is equal to the dual solution (say, some value, x) then that means:

- x is a lower bound on optimum of primal

- x is an upper bound on optimum of dual

If the optimum of the dual was lower than x , then that optimum is a feasible solution which means that optimum is a lower bound on the primal (and is less than x). This is a contradiction.

Similarly, by contradiction, the optimum of the primal can't be larger than x . In other words, x is the optimal solution in this case.

LP-DUALITY (2)

This is a central theorem of Linear Programming called the LP-Duality Theorem.

In fact, not only will this happen if the optimum of the dual is equal to that of the primal but if there is a solution, then it will be equal to both functions.

LP-DUALITY THEOREM

Refer to Primal-Dual Equivalency Definitions

The primal program has finite optimum iff the dual has finite optimum.

Moreover, if $\mathbf{x}^$ and $\mathbf{y}^*$ are optimal solutions for the primal and dual respectively, then*

$$\sum_{j=1}^n c_j x_j^* = \sum_{i=1}^n b_i y_i^*$$

WEAK DUALITY THEOREM

If $\mathbf{x}$ is a feasible solution for the primal and $\mathbf{y}$ is a feasible solution for the dual, then

$$\sum_{j=1}^n c_j x_j \geq \sum_{i=1}^n b_i y_i$$

INTEGER PROGRAMMING (IP)

Integer Programming is simply Linear Programming with an added condition

All variables must be integers

Many problems can be stated as Integer Programs.

For example, the Set Cover problem can be stated as an integer program.

WEIGHTED SET COVER (WSC) AS IP

For each subset we will give an integer variable, 0 or 1, that is 1 if the subset is part of the cover, and 0 if not.

Then we can state the weighted set cover as:

$$\begin{array}{ll}\text{minimize} & \sum_{j=1}^n w_j \cdot x_j \\ \text{subject to :} & \forall_{t_i \in T}, \sum_{j: t_i \in S_j} x_j \geq 1 \\ & \forall_{i \in (1, \dots, m)}, x_j \in \{0, 1\}\end{array}$$

RELAXED LP FOR WSC

We can write the IP to the WSC in a more relaxed form:

$$\begin{array}{ll}\text{minimize} & \sum_{j=1}^n w_j \cdot x_j \\ \text{subject to :} & \forall_{t_i \in T}, \sum_{j: t_i \in S_j} x_j \geq 1 \\ & \forall_{i \in (1, \dots, m)}, x_j \geq 0\end{array}$$

Now, if Z_{LP} is the optimum of this LP, then $Z_{LP} \leq OPT$, because any feasible solution for the IP is also feasible for the LP. So the optimal LP will not be greater than the optimal IP.

USING RELAXED LP AS APPROXIMATION

Given the LP, the optimum is Z_{LP} .

If there is a solution of cost no more than $\alpha \bullet Z_{LP}$, then the cost is no more than $\alpha \bullet OPT$.

Therefore, if the LP is solved for a feasible solution in polynomial time that is α times the optimum LP value, then that is also an α -approximation of the IP, and corresponding Set Cover problem.

BASIC TECHNIQUE

1. Write the IP describing the problem.
2. Relax the IP to get an LP.
3. Find the optimal solution to the LP.
4. Find integral values for the (linear) variables such that the solution is at most α of the optimal solution to the IP.
5. Reformulate the integral values as solution to the problem.

OVERVIEW OF BASIC TECHNIQUE

Steps 1 and 2 can be performed in polynomial time, easily.

Step 5 can also be performed in polynomial time.

Step 3 can be done in polynomial time too, but sometimes it's easier to connect it to step 4.

The tricky part is really step 4, and four ways will be shown to accomplish it.

The methods shown will solve the Set Cover, in particular, but can be used for any IP.

METHOD 1: ROUNDING

This implements step 4 only:

1. $I \leftarrow 0$
2. for each S_j
 3. if $x_j^* \geq \frac{1}{f}$
 4. $I \leftarrow I \cup \{j\}$

PROOF: ROUNDING METHOD PRODUCES SET COVER

Assume by contradiction that there is an element t_i such that

Then,

$$t_i \notin \bigcup_{j \in I} S_j$$

And therefore:

Since,

$$\forall_{S_j: t_i \in S_j} x_j^* < \frac{1}{f}$$

But this violates the LP constraints.

$$\sum_{j: t_i \in S_j} x_j^* < \frac{1}{f} \left| \{j : t_i \in S_j\} \right| \leq 1$$

$$\left| \{j : t_i \in S_j\} \right| < f = \max_{t_i \in T} \left| \{j : t_i \in S_j\} \right|$$

PROOF: ROUNDING IS F -APPROXIMATION ALGORITHM

The algorithm is a polynomial time algorithm for step 4 in the Basic Technique.

Furthermore,

$$\begin{aligned}\sum_{j \in I} w_j &\leq \sum_j w_j w_j^* f \\ &= f \sum_j w_j w_j^* \\ &\leq f \cdot OPT\end{aligned}$$

The first inequality holds, since $j \in I$ only if $x_j^* f \geq 1$

METHOD 2: DUAL-LP

This second method is based on the dual solution.

By weak duality, if there is a feasible solution y , then

$$\sum_i y_i \leq z_{LP} \leq OPT$$

DUAL LP-ALGORITHM

First, solve the dual linear program to get an optimal solution y^*

Then, apply the following algorithm:

1. $I \leftarrow 0$
2. for each S_j
3. if $\sum_{i: t_i \in S_j} y_i^* = w_j$
4. $I \leftarrow I \cup \{j\}$

METHOD 3: GREEDY ALGORITHM

$$I \leftarrow 0$$

$$\forall j, \tilde{S}_j \leftarrow S_j$$

$$\text{while } T \neq \bigcup_{j \in I} S_j$$

$$l \leftarrow \arg \min_{j: \tilde{S}_j \neq 0} \frac{w_j}{|\tilde{S}_j|}$$

$$I \leftarrow I \cup \{l\}$$

$$\forall t_i \in \tilde{S}_l, \tilde{y}_i \leftarrow \frac{w_l}{|\tilde{S}_l| H_g} \text{ (used in proof)}$$

$$\forall j, \tilde{S}_j \leftarrow \tilde{S}_j - S_l$$

GREEDY ALGORITHM

Define

$$H_n \equiv 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots + \frac{1}{n} \approx \ln n$$
$$g \equiv \max_j |S_j|$$

The traveling salesman problem with triangle inequality

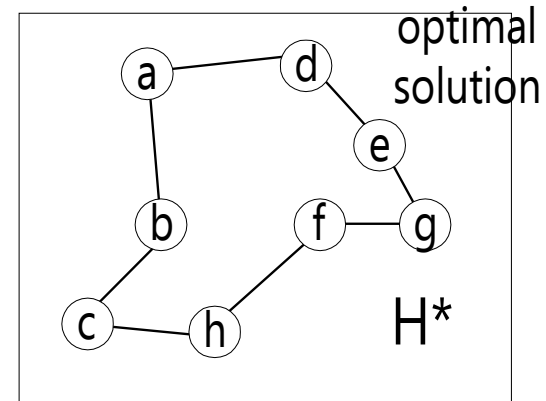
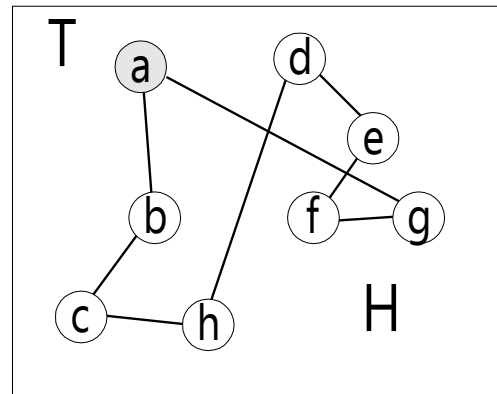
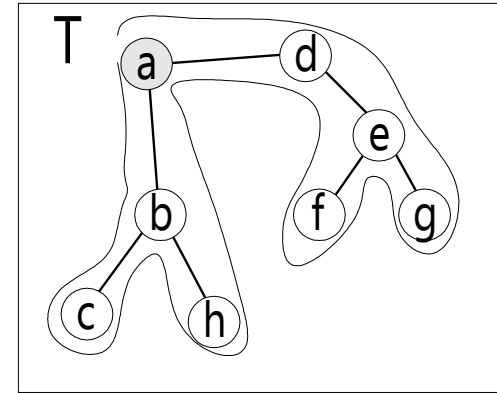
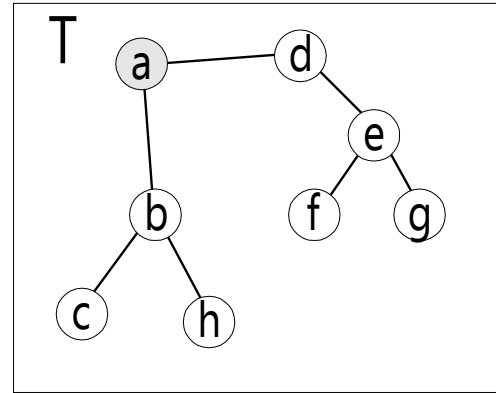
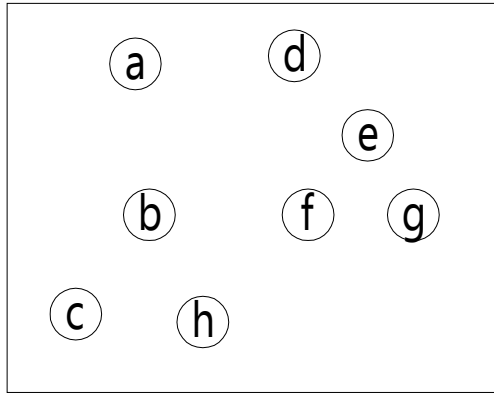
- triangle inequality for the cost matrix

$$c(u, w) \leq c(u, v) + c(v, w) \quad \forall u, v, w$$

35.2.1 The TSP with triangle inequality

APPROX_TSP_TOUR(G, c)

- 1 Select a vertex $r \in V[G]$ to be a root vertex
- 2 grow a MST T for G from root r using
MST_PRIM(G, c, r)
- 3 Let L be the list of vertices visited in a preorder walk of T .
- 4 **return** the hamiltonian cycle H that visit the vertices in the order L .



Theorem 35.2. APPROX_TSP_TOUR is an approximation algorithm with ratio bound of 2 for TSP with triangular inequality.

Proof.

$$c(T) \leq c(H^*)$$

$$c(W) = 2c(T) \leq 2c(H^*)$$

$$c(H) \leq c(W)$$

$$\Rightarrow c(H) \leq 2c(H^*)$$

THE BOTTLENECK TRAVELING SALESPERSON PROBLEM (BTSP)

Minimize the longest edge of a tour.

This is a [mini-max](#) problem.

This problem is NP-hard.

The input data for this problem fulfill the following assumptions:

- The graph is a complete graph.

- All edges obey the [triangular inequality rule](#).

AN ALGORITHM FOR FINDING AN OPTIMAL SOLUTION

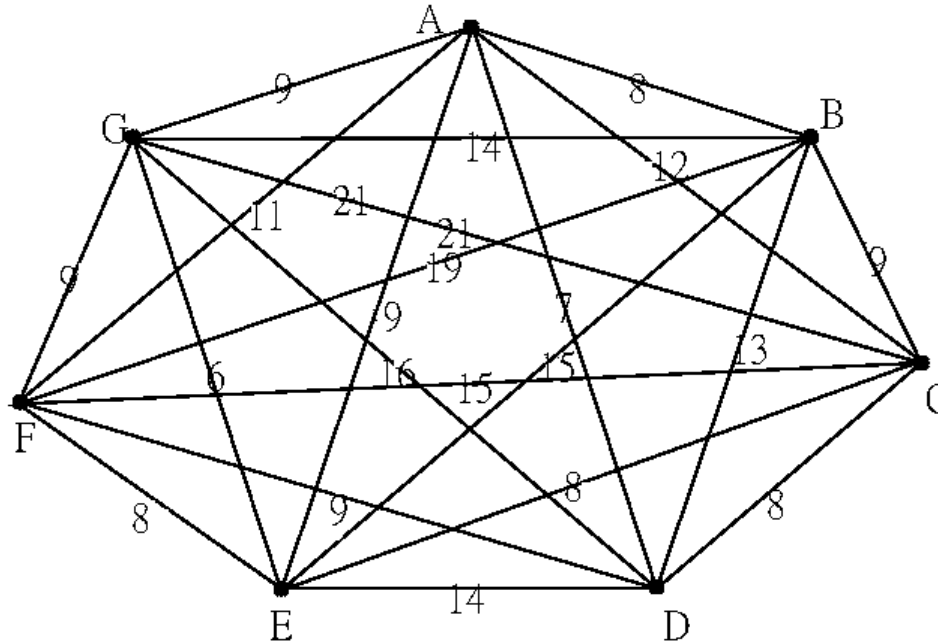
Step1: Sort all edges in $G = (V, E)$ into a nondecreasing sequence $|e_1| \leq |e_2| \leq \dots \leq |e_m|$. Let $G(e_i)$ denote the subgraph obtained from G by deleting all edges longer than e_i .

Step2: $i \leftarrow 1$

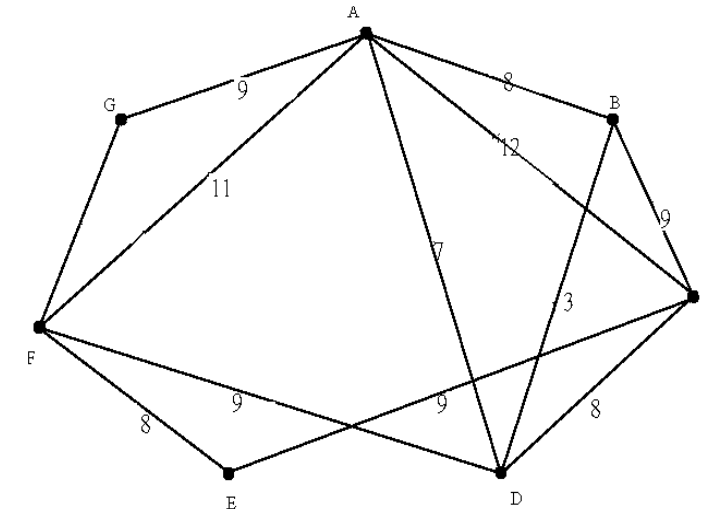
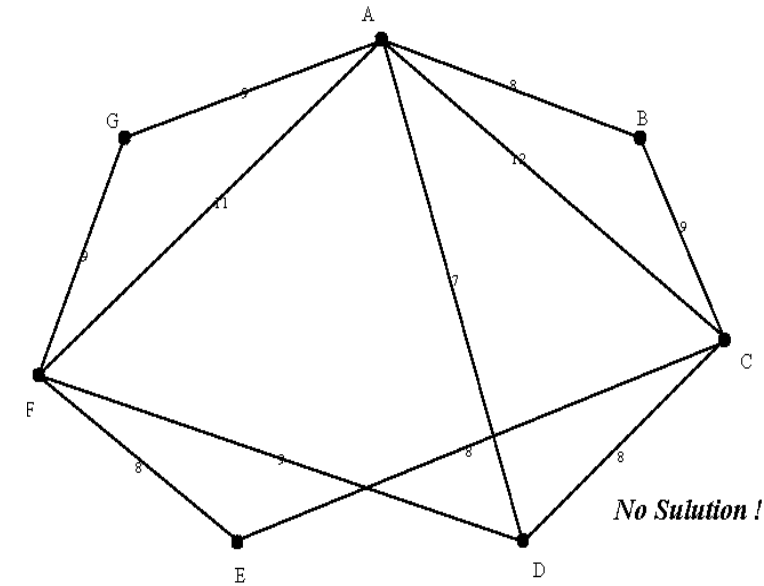
Step3: If there exists a Hamiltonian cycle in $G(e_i)$, then this cycle is the solution; hence, stop.

Step4: $i \leftarrow i + 1$. Go to Step 3.

AN EXAMPLE FOR BTSP ALGORITHM



- There is a Hamiltonian cycle, A-B-D-C-E-F-G-A, in $G(BD)$.
- The optimal solution is 13.



TIME COMPLEXITY OF THE ALGORITHM

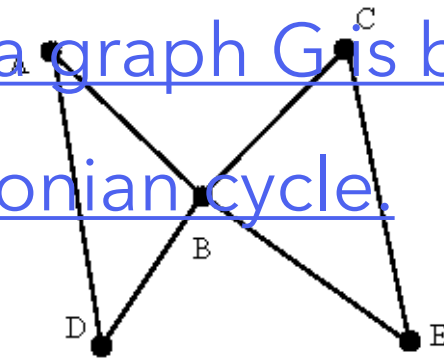
The Hamiltonian cycle problem is NP-hard.

The algorithm cannot be a polynomial one.

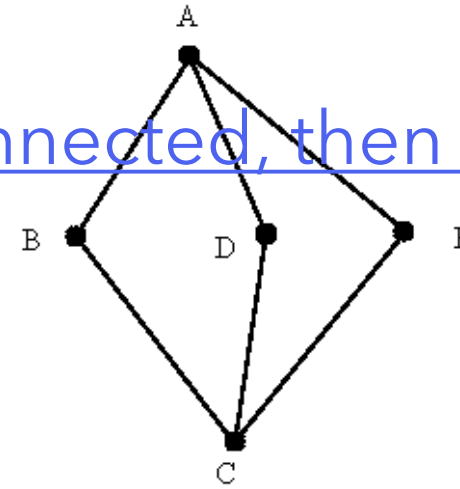
THEOREM FOR HAMILTONIAN CYCLES

Def: The t-th power of $G=(V,E)$, denoted as $G^t=(V,E^t)$, is a graph that an edge $(u,v) \in E^t$ if there is a path from u to v with at most t edges in G .

Theorem: If a graph G is bi-connected, then G^2 has a Hamiltonian cycle.

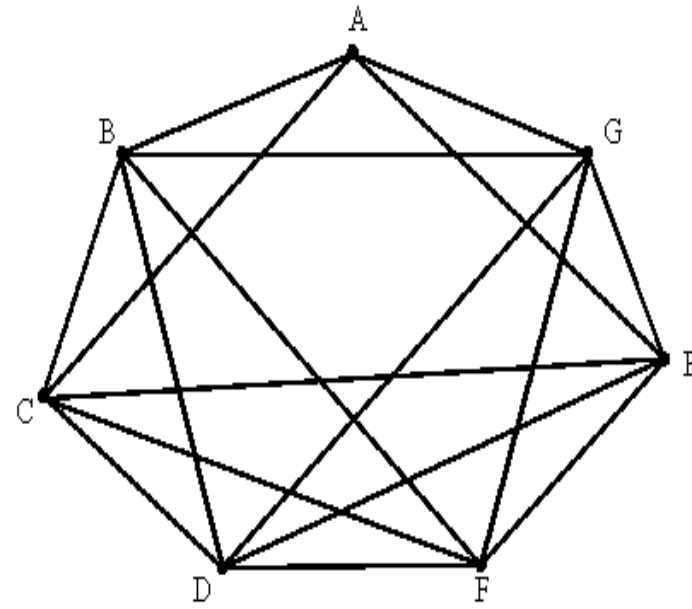
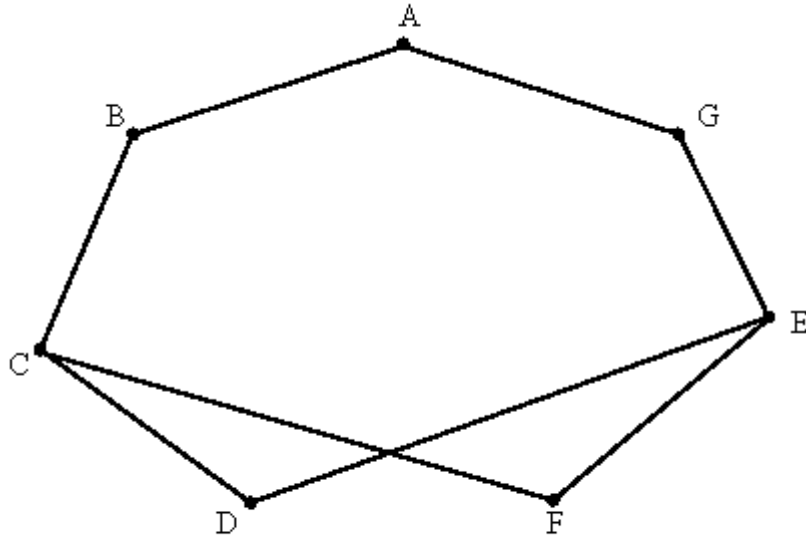


G_a
not bi-connected



G_b
bi-connected

AN EXAMPLE FOR THE THEOREM



G^2

A Hamiltonian cycle:

A-B-C-D-E-F-G-A

AN APPROXIMATION ALGORITHM FOR BTSP

Input: A complete graph $G=(V,E)$ where all edges satisfy triangular inequality.

Output: A tour in G whose longest edge is not greater than twice of the value of an optimal solution to the special bottleneck traveling salesperson problem of G .

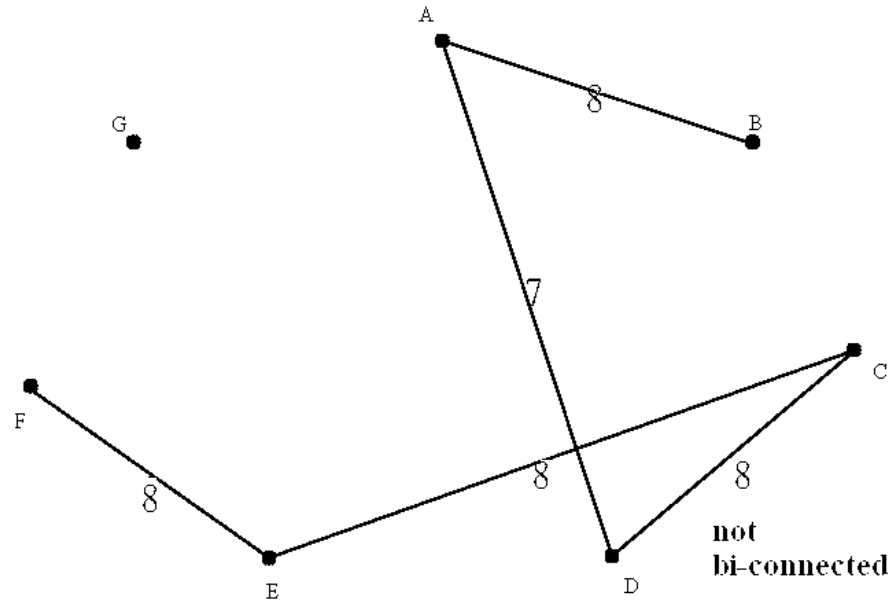
Step 1: Sort the edges into $|e_1| \leq |e_2| \leq \dots \leq |e_m|$.

Step 2: $i := 1$.

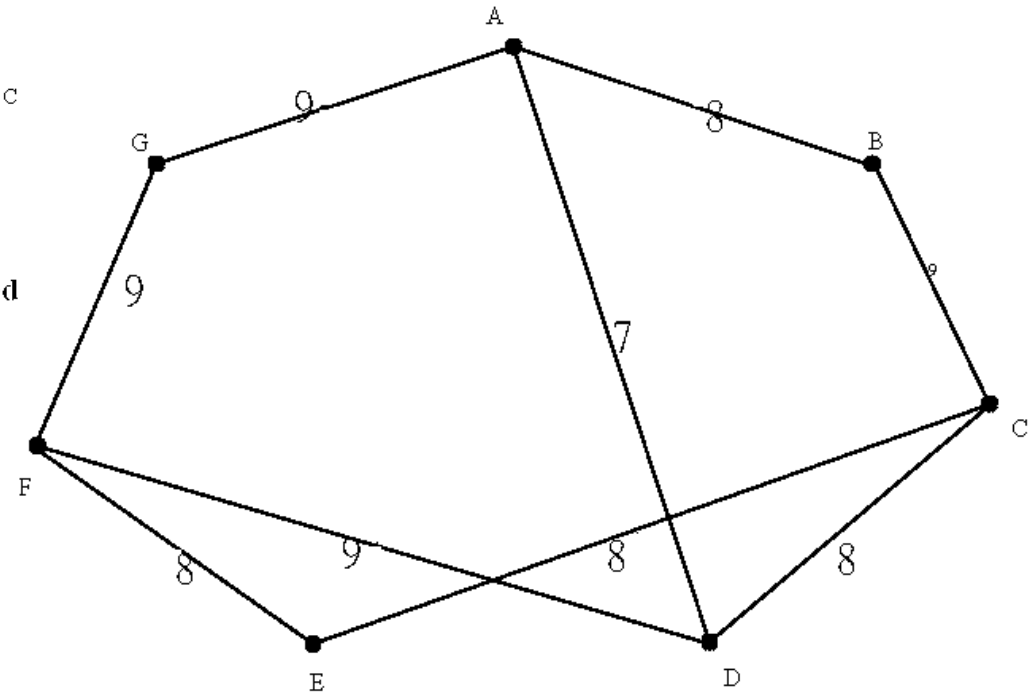
Step 3: If $G(e_i)$ is bi-connected, construct $G(e_i)^2$, find a Hamiltonian cycle in $G(e_i)^2$ and return this as the output.

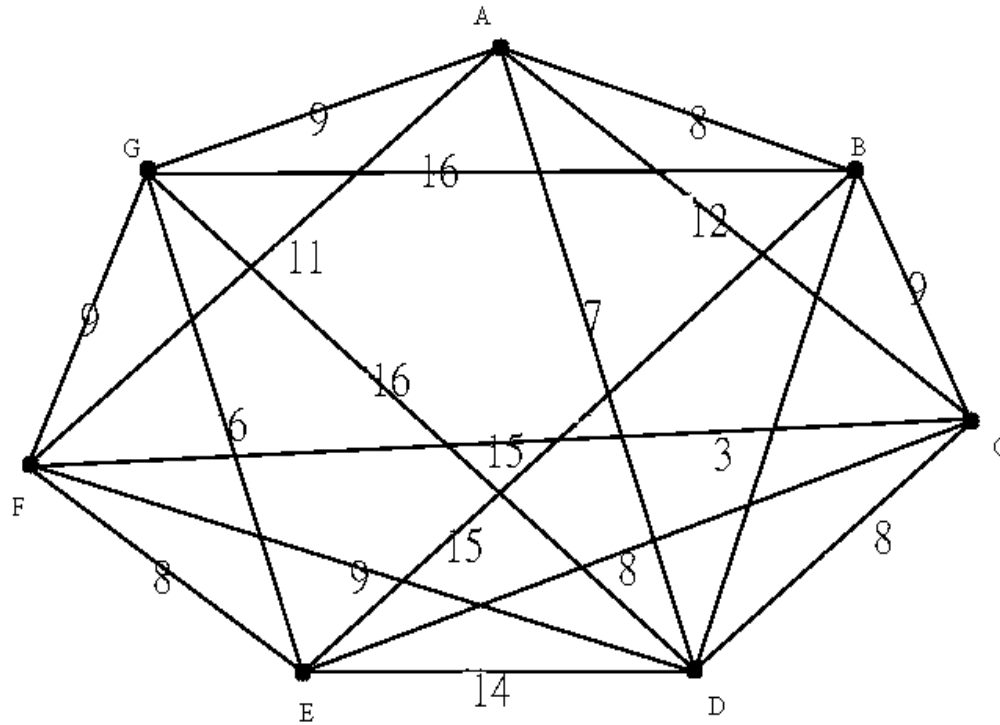
Step 4: $i := i + 1$. Go to Step 3.

AN EXAMPLE



Add some more edges.
Then it becomes bi-connected.





A Hamiltonian cycle: A-G-F-E-D-C-B-A.

The longest edge: 16

TIME COMPLEXITY

The algorithm is of polynomial time

The determination of bi-connectedness can be solved by polynomial algorithms.

If G is bi-connected, then there exists a polynomial algorithm to solve the Hamiltonian cycle problem of G^2 .

HOW GOOD IS THE SOLUTION ?

The value of an approximate solution is bounded by two times that of an optimal solution.

Reasoning:

A Hamiltonian cycle is bi-connected.

e_{op} : the longest edge of an optimal solution

$G(e_i)$: the first bi-connected graph

$$|e_i| \leq |e_{op}|$$

The length of the longest edge in $G(e_i)^2 \leq 2|e_i|$

$$\text{(triangular inequality)} \quad \leq 2|e_{op}|$$



THANK YOU

Module 7

BCSE204L - Design and Analysis of Algorithm